Very Large Arrays and Generators

Imagine you have a use case involving arbitrarily large (or unknown) amounts of data and your system is constrained by memory. Let's say you attempted to iterate over a large array of integers like this:

```
$arr = range(0, 1000000);

foreach ($arr as $val)  {
    // do something
}

print "Peak memory usage " .  memory_get_peak_usage() . "\n";

Output:
Peak memory usage 17169304
```

Here we create an array of one million sequential integers and iterate over it. (It's not important what the `foreach` does here; we just want to consume memory.) Nearly 34 megabytes have been allocated during script execution. What if I told you there was a way to do this that only consumed around five-hundred kilobytes of peak memory?

To mitigate scenarios like this, PHP introduced a new language feature called `generators` in version 5.5. A generator is a special function that does not use the `return` keyword; instead, it uses `yield` and will generate a value each time it is iterated.

In our example above, we'll replace PHP's built-in `range()` – which is currently building an array of one million numbers – with a generator:

```
function gen_range(int $start, int $limit) {
    for  ($i = $start; $i <= $limit; $i++)  {
        yield $i;
    }
}

foreach (gen_range(0,1000000) as $val) {
    // do something
}

print "Peak memory usage " .  memory_get_peak_usage() . "\n";

Output:
Peak memory usage 425776
```